```
pip install numpy scikit-learn
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=1.2.0->scikit-learn
```

```
import numpy as np

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

data = load_breast_cancer()
X = data.data
y = data.target
feature_names = np.array(data.feature_names)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

n_features = X_train.shape[1]
fitness_cache = {}

def fitness(solution):
    mask = np.asarray(solution, dtype=bool)

    if not np.any(mask):
        mask[np.random.randint(n_features)] = True

    key = tuple(mask)

    if key in fitness_cache:
        return fitness_cache[key]

    model = DecisionTreeClassifier(random_state=42)

    scores = cross_val_score(
        model,
        X_train[:, mask],
        y_train,
        cv=3,
        scoring="accuracy"
    )

    score = 1 - scores.mean() + 0.01 * mask.sum() / n_features
    fitness_cache[key] = score

    return score

def cuckoo_search(n_nests=12, iterations=15,
                  abandon_rate=0.25, alpha=0.15):

    nests = np.random.rand(n_nests, n_features)

    def decode(position):
        mask = position > 0.5

        if not np.any(mask):
            mask[np.argmax(position)] = True

        return mask

    scores = np.array([fitness(decode(nest)) for nest in nests])

    best_idx = np.argmin(scores)
    best_nest = nests[best_idx].copy()
    best_score = scores[best_idx]

    for iteration in range(iterations):
        for i in range(n_nests):
            u = np.random.normal()
            v = np.random.normal()
            levy_step = u / (abs(v) ** (1 / 1.5) + 1e-8)
```

```
            candidate = nests[i] + alpha * levy_step * (
                nests[i] - best_nest
            )
            candidate = np.clip(candidate, 0, 1)

            candidate_score = fitness(decode(candidate))

            if candidate_score < scores[i]:
                nests[i] = candidate
                scores[i] = candidate_score

        n_abandon = max(1, int(abandon_rate * n_nests))
        worst_indices = np.argsort(scores)[-n_abandon:]

        for i in worst_indices:
            nests[i] = np.random.rand(n_features)
            scores[i] = fitness(decode(nests[i]))

        best_idx = np.argmin(scores)

        if scores[best_idx] < best_score:
            best_score = scores[best_idx]
            best_nest = nests[best_idx].copy()

        print(
            f"Iteration {iteration + 1}: "
            f"Fitness={best_score:.4f}, "
            f"Features={decode(best_nest).sum()}"
        )

    return decode(best_nest)

selected_mask = cuckoo_search()
selected_features = feature_names[selected_mask]

print("\nSelected Features:")
print(selected_features)
print("Number of selected features:", selected_mask.sum())
print("Total features:", n_features)

model = DecisionTreeClassifier(random_state=42)

model.fit(X_train[:, selected_mask], y_train)
y_pred = model.predict(X_test[:, selected_mask])

print("\nDecision Tree Performance")
print(f"Accuracy:  {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred):.4f}")
print(f"F1-score:  {f1_score(y_test, y_pred):.4f}")


class_names = {0: "Malignant", 1: "Benign"}

print("\nTumor Type Prediction")
print("-" * 45)

for i in range(10):
    actual = class_names[y_test[i]]
    predicted = class_names[y_pred[i]]

    print(f"Sample {i + 1}:")
    print(f"Actual Tumor Type: {actual}")
    print(f"Predicted Tumor Type: {predicted}")
    print(f"Result: {'Correct' if actual == predicted else 'Incorrect'}")
    print("-" * 45)
```

```
Predicted Tumor Type: Benign
Result: Correct
--------------------------------------------
Sample 3:
Actual Tumor Type: Malignant
Predicted Tumor Type: Malignant
Result: Correct
--------------------------------------------
Sample 4:
Actual Tumor Type: Benign
Predicted Tumor Type: Benign
Result: Correct
--------------------------------------------
Sample 5:
Actual Tumor Type: Malignant
Predicted Tumor Type: Malignant
Result: Correct
--------------------------------------------
Sample 6:
Actual Tumor Type: Benign
Predicted Tumor Type: Benign
Result: Correct
--------------------------------------------
Sample 7:
Actual Tumor Type: Benign
Predicted Tumor Type: Benign
Result: Correct
--------------------------------------------
Sample 8:
Actual Tumor Type: Malignant
Predicted Tumor Type: Malignant
Result: Correct
--------------------------------------------
Sample 9:
Actual Tumor Type: Malignant
Predicted Tumor Type: Malignant
Result: Correct
--------------------------------------------
Sample 10:
Actual Tumor Type: Malignant
Predicted Tumor Type: Malignant
Result: Correct
--------------------------------------------
```